



Panchip Microelectronics Co., Ltd.

PAN3029/3060 Driver Interface User Manual

Current Version: 1.6

Release Date: June 2025

Shanghai Panchip Microelectronics Co., Ltd.

Address: Room 302, Building D, 666 Shengxia Road, Zhangjiang Hi-Tech Park, Shanghai

Tel: 021-50802371

Website: <http://www.panchip.com>

Document Description

This document may be updated from time to time due to version upgrades or other reasons. Unless otherwise agreed upon, the contents of this document are for guidance only. All statements, information, and recommendations herein do not constitute any warranty, either express or implied.

Trademark

Panchip is a trademark of Panchip Microelectronics Co., Ltd. Other names mentioned in this document are trademarks or registered trademarks of their respective owners.

Disclaimer

All or part of the products, services, or features described in this document may not be within the scope of your purchase or use. Unless otherwise agreed upon in the contract, Panchi Microelectronics Co., Ltd. makes no representations or warranties, express or implied, with respect to the contents of this document.

Revision History

Version	Revision Date	Description
V1.1	June 2023	Initial version creation
V1.2	August 2023	1. Modified the operation flow after using single-packet mode 2. Added an interface for obtaining the current channel energy detection
V1.3	September 2023	1. Updated the transmit power table 2. Updated the MAPM interface
V1.4	December 2023	1. Updated the transmit power table for levels 1-23
V1.5	December 2023	1. Updated SDK porting instructions
V1.6	June 2025	Updated SDK function naming and function optimization

Table of contents

1 Overview.....	1
1.1 Document Purpose.....	1
1.2 Target Audience.....	1
1.3 Document Description.....	1
2 SDK Usage Process.....	2
2.1 Initialization Process.....	2
2.1.1 Interface Functions.....	2
2.1.2 Functional Implementation.....	2
2.1.3 Usage Instructions.....	2
2.2 Parameter Configuration Process.....	3
2.2.1 Interface Functions.....	3
2.2.2 Functional Implementation.....	3
2.2.3 Usage Instructions.....	3
2.3 Transmitting Process.....	4
2.3.1 Interface Functions.....	4
2.3.2 Functional Implementation.....	4
2.3.3 Usage Instructions.....	4
2.4 Receiving Process.....	5
2.4.1 Interface Functions.....	5
2.4.2 Functional Implementation.....	5
2.4.3 Usage Instructions.....	6
3 SDK Interface Description.....	7
3.1 RF_Init.....	7
3.2 RF_EnterDeepsleepState.....	7
3.3 RF_EnterSleepState.....	7
3.4 RF_ExitSleepState.....	8
3.5 RF_SetFreq.....	8
3.6 RF_SetRegulatorMode.....	9
3.7 RF_SetTxPower.....	9
3.8 RF_SetBW.....	9
3.9 RF_SetSF.....	10
3.10 RF_SetCR.....	10
3.11 RF_SetCRC.....	10
3.12 RF_SetLDR.....	11
3.13 RF_SetPreamLen.....	11
3.14 RF_SetSyncWord.....	11
3.15 RF_SetInvertIQ.....	12
3.16 RF_TxSinglePkt.....	12
3.17 RF_EnterContinuousRxState.....	12
3.18 RF_EnterSingleRxWithTimeout.....	13

3.19 RF_GetRecvPayload	13
3.20 RF_GetPktRssi	13
3.21 RF_GetRealTimeRssi	14
3.22 RF_GetIRQFlag	14
3.23 RF_ClrIRQFlag	14
3.24 RF_GetFreq	15
3.25 RF_GetPreamLen	15
3.26 RF_GetTxPower	15
3.27 RF_GetBandWidth	16
3.28 RF_GetSF	16
3.29 RF_GetCRC	16
3.30 RF_GetCR	17
3.31 RF_GetSyncWord	17
3.32 RF_GetLDR	17
3.33 RF_GetOneSymbolTime	18
3.34 RF_GetTxTimeMs	18
3.35 RF_EnableMapm	18
3.36 RF_DisableMapm	19
3.37 RF_ConfigMapm	19
3.38 RF_SetMapmAddr	20
3.39 RF_GetLeftMapmTime	20
3.40 RF_StartTxContinuousWave	20
3.41 RF_StopTxContinuousWave	21
3.42 RF_StartCad	21
3.43 RF_StopCad	21
4 SDK Porting	23
5 Attachments	25
5.1 Transmitter Power Table	25

1 Overview

This document comprehensively describes how to use the software development kit (SDK) interface functions for the PAN3029/PAN3060 series RF chips. The SDK provides a highly abstract API that simplifies the development process for RF communication functions, enabling developers to quickly implement stable and reliable wireless communication solutions.

1.1 Document Purpose

- Detailed description of the interface definitions and usage of each SDK functional module
- Provides a complete workflow for initialization, configuration, and data transmission and reception
- Guides developers in correctly porting the SDK to different hardware platforms

1.2 Target Audience

- Hardware integration engineers
- Embedded software development engineers

1.3 Document Description

File Name	Function Description
pan_rf.h	RF driver code header file, containing all API interface declarations and data type definitions
pan_rf.c	RF driver code source file, containing the RF underlying and service driver function interfaces and implementation procedures
pan_param.h	RF driver parameter header file, containing multiple RF parameter configuration tables

2 SDK Usage Process

This section describes in detail the basic process of using the SDK, including RF initialization, parameter configuration, and data transmission and reception.

2.1 Initialization Process

2.1.1 Interface Functions

```
/**
 * @brief Initializes the RF transceiver to STB3 state after power-up.
 * @return RF_Err_t returns the operation result.
 * - RF_OK: Operation successful.
 * - RF_FAIL: Operation failed.
 * @note: Before calling this function, you must configure the MCU's SPI
 * and related GPIO pins.
 */
RF_Err_t RF_Init(void);
```

2.1.2 Functional Implementation

1. Reset the chip and switch the PAN3029 from Deepsleep to STB3.
2. Set the bus mode to four-wire or three-wire SPI.
3. Program the recommended register default values.
4. Calibrate RF parameters.

```
RF_Calibrate(); /* Calibrate RF-related parameters */
```

5. Configure the chip's AGC function.

```
RF_ConfigAgc(); /* Configure the AGC function */
```

6. Initialize the antenna control pins.

```
RF_InitAntGpio(); /* Initialize the antenna GPIO pins */
```

2.1.3 Usage Instructions

The interface function is called when the chip is first powered on or reset.

2.2 Parameter Configuration Process

2.2.1 Interface Functions

```
/**  
 * @brief Configure RF chip user parameters  
 */  
void RF_ConfigUserParams(void)
```

2.2.2 Functional Implementation

1. Set frequency, Code_Rate, bandwidth, spreading factor, transmit power and CRC

```
RF_SetTxPower(22);  
  
RF_SetFreq(RF_FREQ_DEFAULT); /* Set frequency */  
RF_SetBW(RF_BW_DEFAULT); /* Set bandwidth */  
RF_SetSF(RF_SF_DEFAULT); /* Set spreading factor */  
RF_SetCR(RF_CR_DEFAULT); /* Set channel coding rate */  
RF_SetCRC(RF_CRC_DEFAULT); /* Set CRC checksum */  
RF_SetLDR(RF_LDR_DEFAULT); /* Set low rate mode */  
  
RF_SetPreamLen(RF_PREAMBLE_DEFAULT); /* Set preamble length */  
RF_SetInvertIQ(RF_IQ_INVERT_DEFAULT); /* Set IQ Do not invert */  
RF_SetChipMode(CHIPMODE_MODE0); /* Set the chip mode to MODE0 */
```

2.2.3 Usage Instructions

After chip initialization is complete, call the interface function in STB3 state.

2.3 Transmitting Process

2.3.1 Interface Functions

```
/**
 * @brief Send a single data packet
 * @param <Buffer> Data buffer to be sent
 * @param <Size> Number of data bytes to be sent
 * @note A TX_DONE interrupt will be triggered after the transmission is
 * complete.
 * @note In the transmit completion interrupt, call the RF_EnableLdoPa
 * () function to disable the chip's internal LDO and PA.
 */
void RF_TxSinglePkt(uint8_t *Buffer, uint8_t Size);
```

2.3.2 Functional Implementation

1. Turn on TX_IO

```
RF_TurnonTxAnt(); /* Turn on the transmit antenna */
```

2. Set the transmit mode

```
RF_SetTxMode(RF_TX_MODE_SINGLE); /* Set single transmit mode */
```

3. Set the transmit state

```
RF_WriteReg(0x02, RF_STATE_TX); /* Enter transmit state */
```

4. Write transmit length and data content

```
RF_WritePageReg(1, 0x0C, Size); /* Set transmit data length */
RF_WritePageRegs(0, 0x01, Buffer, Size); /* Write data to the FIFO. RF will
immediately transmit the data after writing. */
```

5. Get the transmit time of this packet (optional)

```
/**
 * @brief Calculate the time to send a packet
 * @param <Size> The size of the transmit packet, in bytes
 * @return The time to send the packet, in milliseconds
 */
uint32_t RF_GetTxTimeMs(uint8_t Size);
```

2.3.3 Usage Instructions

After the chip is initialized and the parameters are configured, the transmitting interface function is called to send data of the specified length. After the chip completes the sending, a transmit completion interrupt is generated and the IRQ pin of the PAN3029/PAN3060 is set from low to high to notify the HOST of the interrupt event.

2.4 Receiving Process

2.4.1 Interface Functions

1. Continuous Receive Function Interface

```
/**
 * @brief Enters the continuous receive state
 * @note After calling this function, the chip will enter the continuous
 * receive state
 * @note This function sets the chip's operating state to MODE_RX
 */
void RF_EnterContinuousRxState(void);
```

2. Single Receive Function Interface with Timeout

```
/**
 * @brief Enters the single receive state with timeout
 * @param <TimeoutMs> Timeout period, in milliseconds
 * Timeout period range: 0-65535ms
 * @note After calling this function, the chip will enter the single
 * receive state with timeout
 * @note This function sets the chip's operating state to MODE_RX
 */
void RF_EnterSingleRxWithTimeout(uint16_t TimeoutMs);
```

2.4.2 Functional Implementation

1. Turn on the antenna

```
RF_TurnonRxAnt(); /* Turn on the receiving antenna */
```

2. Set the receive timeout

```
RF_SetRxTimeout(TimeoutMs); /* Set the receive timeout */
```

3. Set the receive mode

```
RF_SetRxMode(RF_RX_MODE_SINGLE_TIMEOUT); /* Set the receive mode to single-
shot mode */
```

4. Enter the receive state

```
RF_SetRfState(RF_STATE_RX); /* Enter the receive state */
```

2.4.3 Usage Instructions

After the PAN3029/PAN3060 completes initialization, it calls the receive function interface to receive data. Upon receiving data, the RF generates a receive completion interrupt and sets the IRQ pin from low to high to notify the host of the interrupt event. The host can use polling or interrupts to determine whether a PAN3029/PAN3060 interrupt has occurred. The host uses RF_GetIRQFlag() to determine if it is a receive interrupt and then uses the uint8_t RF_GetRecvPayload(uint8_t *Buffer) function interface to read the length and content of the received data. Additionally, if the user wishes to obtain the signal strength and SNR of the received data packet, use the following interface:

```
/**
 * @brief Gets the RSSI value of the received packet
 * @return RSSI value
 * @note RSSI value range: -128 to 0, unit: dBm
 * @note This function must be called before the receive interrupt is
 * cleared, as clearing the receive interrupt will clear the signal strength
 * register.
 */
int8_t RF_GetPktRssi(void);

/**
 * @brief Gets the SNR value of the received packet
 * @return SNR value
 * @note This function must be called before the receive interrupt is
 * cleared, as clearing the receive interrupt will clear the SNR register.
 * @note SNR value range: -20 to 10, unit: dB
 */
float RF_GetPktSnr(void);
```

3 SDK Interface Description

This section describes in detail the user interface functions provided in the SDK pan_rf.c. Users can select the function interfaces as needed. When performing parameter configuration, users need to switch the PAN3029/PAN3060 to the STB3 state.

3.1 RF_Init

Item	Description
Function Description	RF Initialization
Function Prototype	RF_Err_t RF_Init(void)
Input Parameters	None
Return Value	RF_OK or RF_FAIL
Usage notes	This function must be called after power-up or in deepsleep state.

3.2 RF_EnterDeepsleepState

Item	Description
Function Description	Enters deepsleep state from STB3 state.
Function Prototype	void RF_EnterDeepsleepState(void)
Input Parameters	None
Return Value	None
Usage notes	1. This function must be called in STB3 state. 2. In deepsleep state, chip registers will be lost. 3. To wake up, call RF_Init() to reinitialize the chip.

3.3 RF_EnterSleepState

Item	Description
Function Description	Entering the SLEEP state from STB3

Function Prototype	void RF_EnterSleepState(void)
Input Parameters	None
Return Value	None
Usage notes	1. This function must be called in STB3 state. 2. In the SLEEP state, all chip registers are retained; there is no need to reinitialize the chip. Call RF_ExitSleepState() to wake the chip.

3.4 RF_ExitSleepState

Item	Description
Function Description	Enters STB3 state from SLEEP state
Function Prototype	void RF_ExitSleepState(void)
Input Parameters	None
Return Value	None
Usage notes	This function must be called from SLEEP state.

3.5 RF_SetFreq

Item	Description
Function Description	Sets the communication frequency (unit: Hz)
Function Prototype	void RF_SetFreq(uint32_t Frequency)
Input Parameters	Communication Frequency (Hz) Supported frequency range: Low Band: -138.33MHz to 282.5MHz -405.00MHz to 565.00MHz High Band: -810.00MHz to 1080.00MHz
Return Value	RF_OK or RF_FAIL
Usage notes	1. This function must be called in STB3 mode. 2. The SDK uses the 470CN frequency band by default. To change to another frequency band, modify the REGION_DEFAULT macro in pan_rf.h.

3.6 RF_SetRegulatorMode

Item	Description
Function Description	Sets the chip's internal power supply mode
Function Prototype	void RF_SetRegulatorMode(RfRegulatorMode_t RegulatorMode)
Input Parameters	typedef enum { USE_LDO = 0x00, //!< Use LDO for power regulation USE_DCDC = 0x01, //!< Use DCDC for power regulation } RfRegulatorMode_t;
Return Value	None
Usage notes	1. This function must be called in STB3 mode 2. By default, the LDO power supply is used. If using DC-DC mode, the PAN3029/3060 must have a corresponding DC-DC circuit externally; otherwise, it will not function properly.

3.7 RF_SetTxPower

Item	Description
Function Description	Sets the RF transmit power level
Function Prototype	void RF_SetTxPower(uint8_t TxPower)
Input Parameters	Transmit power level, range: 1-22
Return Value	None
Usage notes	1. This function must be called in STB3 mode 2. A power level of 22 corresponds to an actual power of approximately 19dBm.

3.8 RF_SetBW

Item	Description
Function Description	Sets the communication bandwidth
Function Prototype	void RF_SetBW(uint8_t BandWidth)

Input Parameters	<BandWidth> Modulation bandwidth value RF_BW_062K / RF_BW_125K / RF_BW_250K / RF_BW_500K
Return Value	None
Usage notes	This function must be called in STB3 mode

3.9 RF_SetSF

Item	Description
Function Description	Sets the spreading factor
Function Prototype	void RF_SetSF(uint8_t SpreadFactor)
Input Parameters	<SpreadFactor>: Spreading factor value RF_SF5 / RF_SF6 / RF_SF7 / RF_SF8 / RF_SF9 / RF_SF10 / RF_SF11 / RF_SF12
Return Value	None
Usage notes	This function must be called in STB3 mode

3.10 RF_SetCR

Item	Description
Function Description	Sets the communication coding rate
Function Prototype	void RF_SetCR(uint8_t CodingRate)
Input Parameters	<CodingRate> Channel coding rate value RF_CR_4_5 / RF_CR_4_6 / RF_CR_4_7 / RF_CR_4_8
Return Value	None
Usage notes	This function must be called in STB3 mode

3.11 RF_SetCRC

Item	Description
Function Description	Sets the CRC checksum for the data packet
Function Prototype	void RF_SetCRC(uint8_t CrcMode)
Input Parameters	<CrcMode>: Enables or disables the CRC checksum RF_CRC_ON: Enables the CRC checksum RF_CRC_OFF: Disables the CRC checksum

Return Value	None
Usage notes	This function must be called in STB3 mode

3.12 RF_SetLDR

Item	Description
Function Description	Sets the low-rate optimization mode
Function Prototype	void RF_SetLDR(uint8_t LdrMode)
Input Parameters	<LdrMode>: Low-rate mode value RF_LDR_ON: Enables low-rate mode RF_LDR_OFF: Disables low-rate mode
Return Value	None
Usage notes	This function must be called in STB3 mode

3.13 RF_SetPreamLen

Item	Description
Function Description	Sets the preamble length
Function Prototype	void RF_SetPreamLen(uint16_t PreamLen)
Input Parameters	Preamble length, range: 4 to 65535
Return Value	None
Usage notes	This function must be called in STB3 mode

3.14 RF_SetSyncWord

Item	Description
Function Description	Sets the synchronization word
Function Prototype	void RF_SetSyncWord(uint8_t SyncWord)
Input Parameters	Sync word
Return Value	None
Usage notes	1. This function must be called in STB3 mode

	2. Both the transmitter and receiver must set the same synchronization word; otherwise, communication will not function properly.
--	---

3.15 RF_SetInvertIQ

Item	Description
Function Description	Sets IQ inversion
Function Prototype	void RF_SetInvertIQ(bool NewState)
Input Parameters	<NewState> Enables or disables IQ inversion true: Enables IQ inversion false: Disables IQ inversion
Return Value	None
Usage notes	1. This function must be called in STB3 mode 2. Both the transmitter and receiver must set the same IQ mode; otherwise, communication will not work properly.

3.16 RF_TxSinglePkt

Item	Description
Function Description	Sends a single packet of data
Function Prototype	void RF_TxSinglePkt(uint8_t *Buffer, uint8_t Size)
Input Parameters	<Buffer> Buffer of data to be sent <Size> Number of data bytes to be sent
Return Value	None
Usage notes	1. This function must be called in STB3 mode 2. After data transmission is complete, call RF_EnableLdoPa() to disable the chip's internal PA

3.17 RF_EnterContinuousRxState

Item	Description
Function Description	Enters continuous receive mode
Function Prototype	void RF_EnterContinuousRxState(void)
Input Parameters	None

Return Value	None
Usage notes	This function must be called in STB3 mode

3.18 RF_EnterSingleRxWithTimeout

Item	Description
Function Description	Enters the chip into single receive mode with timeout
Function Prototype	void RF_EnterSingleRxWithTimeout(uint16_t TimeoutMs)
Input Parameters	Input Parameters: <TimeoutMs> Timeout period, in milliseconds Timeout range: 1 to 65535 milliseconds
Return Value	None
Usage notes	1. This function must be called in STB3 mode 2. After receiving a packet or after a receive timeout, the user must call the receive function again to put the chip into receive mode

3.19 RF_GetRecvPayload

Item	Description
Function Description	Gets the length and content of received data
Function Prototype	uint8_t RF_GetRecvPayload(uint8_t *Buffer)
Input Parameters	<Buffer> Pointer to the data area to be received
Return Value	Received data length
Usage notes	Because clearing the receive interrupt flag will clear the receive length register, this function must be called before the receive interrupt is cleared; otherwise, the received length will be 0.

3.20 RF_GetPktRssi

Item	Description
Function Description	Gets the RSSI value of the received data packet
Function Prototype	int8_t RF_GetPktRssi(void)
Input Parameters	None

Return Value	The RSSI value of the data packet
Usage notes	Because clearing the receive interrupt flag will clear the signal strength register, this function must be called before the receive interrupt flag is cleared.

3.21 RF_GetRealTimeRssi

Item	Description
Function Description	Gets the real-time RSSI value
Function Prototype	int8_t RF_GetRealTimeRssi(void)
Input Parameters	None
Return Value	Real-time RSSI value over the air
Usage notes	Ensure the RF is in receive mode before calling this function

3.22 RF_GetIRQFlag

Item	Description
Function Description	Gets the interrupt flag
Function Prototype	uint8_t RF_GetIRQFlag(void)
Input Parameters	None
Return Value	IRQ flag
Usage notes	

3.23 RF_ClrIRQFlag

Item	Description
Function Description	Clears the interrupt flag
Function Prototype	void RF_ClrIRQFlag(uint8_t IRQFlag)
Input Parameters	<IRQFlag> Interrupt flag
Return Value	None
Usage notes	1. Clearing the receive interrupt will clear the receive length register.

	2. Clearing the receive interrupt will clear the PktRssi register. 3. Clearing the receive interrupt will clear the SNR register. 4. The MAPM interrupt flag does not need to be cleared by software; the chip automatically clears the MAPM interrupt flag after one symbol.
--	--

3.24 RF_GetFreq

Item	Description
Function Description	Gets the currently set communication frequency
Function Prototype	uint32_t RF_GetFreq(void)
Input Parameters	None
Return Value	Current communication frequency
Usage notes	

3.25 RF_GetPreamLen

Item	Description
Function Description	Gets the currently set preamble length
Function Prototype	uint16_t RF_GetPreamLen(void)
Input Parameters	None
Return Value	Current preamble length
Usage notes	

3.26 RF_GetTxPower

Item	Description
Function Description	Gets the currently set transmit power level
Function Prototype	uint8_t RF_GetTxPower(void)
Input Parameters	None
Return Value	Current transmit power level

Usage notes	
-------------	--

3.27 RF_GetBandWidth

Item	Description
Function Description	Gets the currently set communication bandwidth
Function Prototype	uint8_t RF_GetBandWidth(void)
Input Parameters	None
Return Value	Current communication bandwidth
Usage notes	

3.28 RF_GetSF

Item	Description
Function Description	Gets the currently set spreading factor
Function Prototype	uint8_t RF_GetSF(void)
Input Parameters	None
Return Value	Current spreading factor
Usage notes	

3.29 RF_GetCRC

Item	Description
Function Description	Gets the currently set CRC mode
Function Prototype	uint8_t RF_GetCRC(void)
Input Parameters	None
Return Value	Current CRC mode
Usage notes	

3.30 RF_GetCR

Item	Description
Function Description	Gets the currently set coding rate
Function Prototype	uint8_t RF_GetCR(void)
Input Parameters	None
Return Value	Current coding rate
Usage notes	

3.31 RF_GetSyncWord

Item	Description
Function Description	Gets the currently set synchronization word
Function Prototype	uint8_t RF_GetSyncWord(void)
Input Parameters	None
Return Value	Current synchronization word
Usage notes	

3.32 RF_GetLDR

Item	Description
Function Description	Gets the currently set low-rate optimization mode
Function Prototype	uint8_t RF_GetLDR(void)
Input Parameters	None
Return Value	Current low-rate optimization mode
Usage notes	

3.33 RF_GetOneSymbolTime

Item	Description
Function Description	Calculates the time required to transmit a single symbol in us
Function Prototype	uint32_t RF_GetOneSymbolTime(uint8_t bw, uint8_t sf)
Input Parameters	<bw> Bandwidth RF_BW_062K / RF_BW_125K / RF_BW_250K / RF_BW_500K <sf> Spreading Factor RF_SF5/RF_SF6/RF_SF7/RF_SF8/RF_SF9/RF_SF10/RF_SF11/ RF_SF12
Return Value	Single symbol time
Usage notes	

3.34 RF_GetTxTimeMs

Item	Description
Function Description	Calculates the time required to transmit a packet in milliseconds
Function Prototype	uint32_t RF_GetTxTimeMs(uint8_t Size)
Input Parameters	<Size> Size of the sent data packet, in bytes
Return Value	Time to send the data packet, in milliseconds
Usage notes	

3.35 RF_EnableMapm

Item	Description
Function Description	Enables MAPM mode
Function Prototype	void RF_EnableMapm(void)
Input Parameters	None
Return Value	None
Usage notes	This function must be called in STB3 mode

3.36 RF_DisableMapm

Item	Description
Function Description	Disables MAPM mode
Function Prototype	void RF_DisableMapm(void)
Input Parameters	None
Return Value	None
Usage notes	This function must be called in STB3 mode

3.37 RF_ConfigMapm

Item	Description
Function Description	Configures MAPM related parameters
Function Prototype	void RF_ConfigMapm(RF_MapmCfg_t *pMapmCfg)
Input Parameters	<pMapmCfg> fn: Number of fields in the MAPM fnm: Number of times the same field is sent repeatedly (default: fnm=0). gfs: The payload function of the last group in each field - When gfs=0, the payload function of the last group is the address - When gfs=1, the payload function of the last group is the number of remaining fields (including the current field) gn: Number of groups in a field pg1: Number of preambles in the first group of the field, range: 8-255 - When pg1 = 8, it indicates that the first group contains 8 preambles - When pg1 = 200, it indicates that the first group contains 200 preambles. pgn: Number of preambles in the remaining groups of the field, range: 0-255 - When pgn = 8, it indicates that the first group contains 8 preambles - When pgn = 200, it indicates that the first group contains 200 preambles pn: Number of preambles in the packet, range: 1-65535
Return Value	None
Usage notes	This function must be called in STB3 mode

3.38 RF_SetMapmAddr

Item	Description
Function Description	Sets the group address in Mapm mode
Function Prototype	void RF_SetMapmAddr(uint8_t *MapmAddr, uint8_t AddrWidth)
Input Parameters	<MapmAddr> Mapm group address <AddrWidth> Address width, range: 1to 4
Return Value	None
Usage notes	This function must be called in STB3 mode

3.39 RF_GetLeftMapmTime

Item	Description
Function Description	Gets the remaining MAPM time in MAPM mode
Function Prototype	uint32_t RF_GetLeftMapmTime(RF_MapmCfg_t *pMapmCfg, uint32_t SymbolTime)
Input Parameters	<pMapmCfg> MAPM configuration parameters <SymbolTime> Single symbol (chirp) time
Return Value	Remaining Mapm time
Usage notes	

3.40 RF_StartTxContinuousWave

Item	Description
Function Description	Starts transmitting continuous carrier wave
Function Prototype	void RF_StartTxContinuousWave(void)
Input Parameters	None
Return Value	None
Usage notes	1. This function must be called in STB3 mode 2. After calling this function, the chip will remain in the transmitting

	state until the RF_StopTxContinuousWave() function is called to stop transmitting
--	---

3.41 RF_StopTxContinuousWave

Item	Description
Function Description	Stops transmitting continuous carrier wave
Function Prototype	void RF_StopTxContinuousWave(void)
Input Parameters	None
Return Value	None
Usage notes	After calling this function, the chip stops transmitting continuous carrier wave and enters the STB3 state

3.42 RF_StartCad

Item	Description
Function Description	Starts CAD detection
Function Prototype	void RF_StartCad(uint8_t Threshold, uint8_t Chirps)
Input Parameters	<Threshold> - RF_CAD_THRESHOLD_0A - RF_CAD_THRESHOLD_10 - RF_CAD_THRESHOLD_15 - RF_CAD_THRESHOLD_20 <Chirps> - RF_CAD_01_SYMBOL - RF_CAD_02_SYMBOL - RF_CAD_03_SYMBOL - RF_CAD_04_SYMBOL
Return Value	None
Usage notes	After calling this function, the chip enters the receive state and begins detecting CAD signals

3.43 RF_StopCad

Item	Description
Function Description	Stops CAD detection
Function Prototype	void RF_StopCad(void)



Input Parameters	None
Return Value	None
Usage notes	After calling this function, the RF exits the receive state and returns to the STB3 state

Confidential

4 SDK Porting

This section explains how to adapt the MCU to the RF low-level driver interface when porting the PAN3029/PAN3060 driver. The PAN3029/PAN3060 SDK files "pan_rf.c" and "pan_rf.h" provide detailed interface adaptation annotations. The necessary SDK driver adaptation interfaces are summarized below:

- **SPI chip select high:** Requires adaptation of the SPI_CS_HIGH() macro interface.
- **SPI chip select low:** Requires adaptation of the SPI_CS_LOW() macro interface.
- **SPI single-byte data write:** Requires adaptation of the SPI_WriteByte() function interface.
- **SPI single-byte data read:** Requires adaptation of the SPI_ReadByte() function interface.
- **Millisecond delay function:** Requires adaptation of the RF_DelayMs() function interface.
- **Microsecond delay function:** Requires adaptation of the RF_DelayUs() function interface.
- **RF Interrupt Pin**

Connect the PAN3029/PAN3060 interrupt (IRQ) pin to a GPIO on the MCU. The MCU can use interrupts or queries to receive RF interrupt events. Active-high is required.

- **RF CAD Pin**

Connect the PAN3029/PAN3060 CAD (GPIO11) pin to a GPIO on the MCU. The MCU can use interrupts or queries to receive CAD events. Active-high is required.

- **Antenna Transmit/Receive Control Pin (Optional)**

If you need to use the PAN3029/PAN3060 GPIO to control the antenna's transmit and receive functions, configure the following macros based on your needs:

```
#define MODULE_GPIO_TX 0 /* Configure GPIO0 as the transmit control pin */
#define MODULE_GPIO_RX 10 /* Configure GPIO10 as the receive control pin */
```

If your MCU has sufficient GPIOs, use the MCU's GPIOs to control the antenna's transmit and receive functions.

- **Crystal Oscillator Control Pin (Optional)**

If you need to use the PAN3029/PAN3060 to control the crystal oscillator, configure the following macros based on your needs:

```
#define MODULE_GPIO_TCXO 3 /* Configure GPIO3 as the crystal oscillator control pin */
```

If your MCU has sufficient GPIOs, use the MCU's GPIOs to control the crystal oscillator.

- **SPI three-wire/four-wire bus mode switching**

In 01_SDK\mcu\bsp\bsp.h, modify the INTERFACE_MODE macro definition to switch between four-wire and three-wire SPI bus modes. The SDK uses a four-wire SPI bus by default.

```
#define INTERFACE_MODE USE_SPI_4LINE /* PAN3029/3060 uses four-wire SPI mode */
```

```
// #define INTERFACE_MODE USE_SPI_3LINE /* PAN3029/3060 uses 3-wire SPI mode */
```

Notes:

1. The maximum supported SPI clock rate is 10MHz.
2. The millisecond and microsecond delay functions should be as accurate as possible, or slightly off the actual time.
3. If the MCU uses interrupts to obtain interrupt events from the PAN3029/3060, if the MAPM function is used, it is recommended to configure the GPIO corresponding to the RF interrupt pin to the rising edge interrupt trigger mode. If the MAPM function is not used, the corresponding GPIO can be configured to the high level interrupt trigger mode.

5 Attachments

5.1 Transmitter Power Table

frequency gear	410MHz		430MHz		450MHz		460MHz	
	Power (dBm)	Current(mA)	Power (dBm)	Current(mA)	Power (dBm)	Current(mA)	Power (dBm)	Current(mA)
1	-18.733	7.267	-18.165	7.418	-18.767	7.546	-19.679	7.607
2	-8.276	9.42	-7.936	9.563	-8.596	9.671	-9.548	9.758
3	1.599	17.935	1.748	18.349	0.947	18.725	-0.111	18.932
4	3.925	20.301	4.513	20.661	4.114	20.967	3.348	21.439
5	4.921	21.76	5.329	22.183	4.79	22.662	3.865	23.076
6	5.413	22.933	5.501	23.385	4.776	23.811	3.841	24.136
7	7.104	26.605	7.611	26.904	7.345	27.468	6.805	28.54
8	7.655	28.062	8.152	28.282	7.951	28.995	7.491	30.298
9	8.638	30.074	9.185	30.663	8.755	31.328	8.117	32.387
10	8.966	32.23	9.29	31.92	9.106	32.679	8.897	34.62
11	10.274	35.676	10.746	36.083	10.571	37.273	10.298	39.578
12	10.816	39.495	10.988	38.391	10.753	38.94	10.56	41.211
13	11.567	41.972	11.815	41.256	11.612	42.165	11.468	44.97
14	12.83	46.616	13.205	46.748	13.077	48.484	12.919	52.149
15	13.782	50.643	14.252	51.595	14.182	54.016	14.001	58.272
16	14.871	55.72	15.404	57.538	15.36	60.671	15.015	64.89
17	15.404	58.355	15.957	60.647	15.797	63.599	15.317	67.146
18	16.052	63.254	16.602	65.475	16.522	69.186	16.267	74.527
19	16.493	66.892	16.956	68.616	16.899	72.499	16.738	78.608
20	17.414	73.954	17.868	75.84	17.8	80.261	17.632	86.882
21	18.229	87.976	18.297	85.286	17.994	86.519	17.694	90.391
22	19.436	102.128	19.414	98.008	19.07	98.768	18.736	102.895
23	19.754	105.378	19.771	101.687	19.433	102.835	19.105	107.278

Note: The power range is the input parameter of the RF_SetTxPower function. The transmit power and transmit current of each frequency band will be slightly different.